Pseudo-code of the neighborhood generation in the Simulated Annealing algorithm

Supplementary material to the paper
Parallel machine scheduling with simultaneous interruptions: a case study in shoe sole production

Solution representation

A feasible solution is represented by a p-tuple $S = (S_1, \ldots, S_p)$, one entry per shelf, where each shelf

$$S_\ell = \big(\sigma_{\ell,1}, \ldots, \sigma_{\ell,|S_\ell|}\big)$$

is the ordered sequence of the *sub-jobs* processed on it, in the order in which they are processed. A sub-job is a triple $\sigma = (j, i, q)$ formed by a job $j \in J$ (a shoe size), one of its mold copies $i \in \{1, \ldots, c_j\}$, and the quantity $q \in \mathbb{Z}_{\geq 0}$ of soles produced with that copy. We write

$$\Sigma(S) = \big\{(\ell, r) : 1 \leq \ell \leq p,\ 1 \leq r \leq |S_\ell|\big\}$$

for the set of positions occupied by the sub-jobs of S, and $S_\ell[r]$ for the sub-job in position r of shelf ℓ.

Every solution generated by the algorithm satisfies the two invariants

(I1) each pair (j, i), with $j \in J$ and $i \in \{1, \ldots, c_j\}$, occurs exactly once in S;

(I2) $\sum_{i=1}^{c_j} q_{j,i} = n_j$ for every job $j \in J$, where $q_{j,i}$ is the quantity of the sub-job (j, i) and n_j is the number of soles of size j required in the order.

The initial solution is built by splitting the quantity n_j of each job over its c_j mold copies with the procedure `SplitUniformly()` of Algorithm 1, and by appending each resulting sub-job to a shelf drawn uniformly at random, jobs being taken in the order in which they appear in the instance and mold copies in increasing order of i. Both operators described below preserve (I1)–(I2), so every candidate solution generated during the search is feasible and no repair step is required.

Neighborhood

A neighbor of the current solution is generated by the procedure `TwoStepNeighbor()` of Algorithm 1, which combines two elementary moves: `Relocate()`, a positional reallocation that moves a single sub-job to a randomly chosen shelf and position without changing any quantity, and `Redistribute()`, a quantity redistribution that resamples how the quantity of a multi-mold job is divided among its mold copies without changing any position. Three points of the procedure deserve to be made explicit.

- The sub-job drawn at the start of `TwoStepNeighbor()` is used *only* to decide which of the two moves is applied. Neither branch necessarily acts on it: `Relocate()` and `Redistribute()` draw their own sub-job, respectively their own job, independently of that first draw.

- The name "two-step" refers to the single-mold branch, in which two independent relocations are applied in sequence, the second one acting on the outcome of the first. The multi-mold branch applies a single redistribution and no relocation.
- In `Redistribute()`, the quantity n_j of the selected job is partitioned uniformly at random over the vectors of c_j non-negative integers summing to n_j, by the classical stars-and-bars scheme (procedure `SplitUniformly()`). Zero parts are admissible, which allows the search to concentrate a job on a subset of its mold copies.

Algorithm 1: Neighbor generation in the SA algorithm

Input: Solution S; number of shelves p; mold availabilities $\{c_j\}_{j \in J}$
Output: A neighbor of S

Function `TwoStepNeighbor`(S)**:**
  Draw (ℓ^*, r^*) uniformly from $\Sigma(S)$; let j^* be the job of $S_{\ell^*}[r^*]$; `// switch only`
  if $c_{j^*} = 1$ **then**
    return `Relocate(Relocate(`S`))` ; `// two independent relocations`
  else
    return `Redistribute(`S`)` ; `// single redistribution`

Function `Relocate`(S)**:**
  $S' \leftarrow$ copy of S;
  Draw (ℓ_s, r_s) uniformly from $\Sigma(S')$; $\sigma \leftarrow S'_{\ell_s}[r_s]$; `// quantity of` σ `unchanged`
  Draw ℓ_t uniformly from $\{1, \ldots, p\}$; `//` $\ell_t = \ell_s$ `allowed`
  Draw r_t uniformly from $\{1, \ldots, |S'_{\ell_t}| + 1\}$; `// indexed before removal`
  Delete σ from S'_{ℓ_s} at position r_s;
  if $\ell_t = \ell_s$ ***and*** $r_t > r_s$ **then** $r_t \leftarrow r_t - 1$;
  Insert σ into S'_{ℓ_t} at position r_t; **return** S';

Function `Redistribute`(S)**:**
  $S' \leftarrow$ copy of S; $M \leftarrow \{\, j \in J : c_j > 1 \,\}$;
  if $M = \emptyset$ **then return** S';
  Draw $j^\dagger$ uniformly from M ; `// independent of` j^*
  $P \leftarrow$ positions of the $c_{j^\dagger}$ sub-jobs of $j^\dagger$, sorted by mold index;
  $(q_1, \ldots, q_{c_{j^\dagger}}) \leftarrow$ `SplitUniformly(`$n_{j^\dagger}$`,` $c_{j^\dagger}$`)` ; `// uniform on` $\{q \geq 0 : \sum_i q_i = n_{j^\dagger}\}$
  for $i = 1$ **to** $c_{j^\dagger}$ **do** set the quantity of the i-th sub-job of P to q_i ; `// positions unchanged`
  ;
  return S';

Function `SplitUniformly`(N, k)**:**
  if $k = 1$ ***or*** $N = 0$ **then return** $(N, 0, \ldots, 0)$;
  Draw $\{d_1 < \cdots < d_{k-1}\}$ uniformly among the $(k-1)$-subsets of $\{1, \ldots, N + k - 1\}$;
  $d_0 \leftarrow 0$; $d_k \leftarrow N + k$; **return** $(d_i - d_{i-1} - 1)_{i=1}^{k}$;

Correspondence with the implementation

Algorithm 1 is a line-by-line transcription of four functions of the file `simulated_annealing.jl`, in the directory `scripts` of the project repository

https://github.com/ndlopes-github/ShoeOptSetupTime

namely:

`TwoStepNeighbor()`	`two_step_neighbor`
`Relocate()`	`neighbor_ppartition`
`Redistribute()`	`neighbor_quantity_redistribution`
`SplitUniformly()`	`split_quantity_randomly`

The initial solution is built by `create_random_partition` in the same file.